

4160 – Distributed and Parallel Computing

P2 – OpenMP



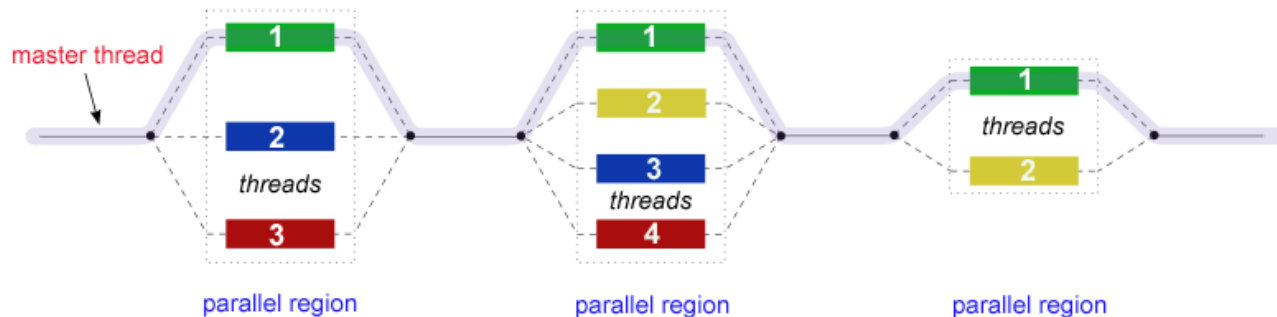
CUHK

Open Multi-Processing

- An API with different implementations
 - gcc, clang, ...
 - Intel implementation is by pthread
- Multi-threaded shared-memory programming model
 - Threads communicate by sharing variables
- Give easy parallelization but no guarantee efficient use of shared memory
- OpenMP supposes for parallelizable jobs
 - No handling race conditions, deadlocks, ... for you
- Parallelism is expressed as threads

Fork-Join Model

- Begin as a single master thread
- Single-thread execution until reaching a **parallel region**, which
 - Fork: a team of parallel threads
 - Join: all team threads complete, synchronize, and terminate



Others

- Data Scoping
 - Shared memory model, data is shared among threads in a parallel region
 - Can have a way to explicitly define scope
- Nested Parallelism
 - Depending on the OpenMP implementation, can support a parallel region nested inside a parallel region
- Dynamic Thread
 - Depending on the OpenMP implementation, can dynamically alter the number of threads used to execute parallel regions
- `#pragma omp simd`
- ...

Creating **Parallel Regions**

```
#include <stdio.h>
#include <omp.h>
int main() {
    ... //serial here
    #pragma omp parallel num_threads(3) { //fork here
        int id = omp_get_thread_num();
        int total = omp_get_num_threads();
        printf("Greetings from process %d out of %d\n", id, total);
    } //join here
    printf("parallel for ends.\n");
    return 0;
}
```

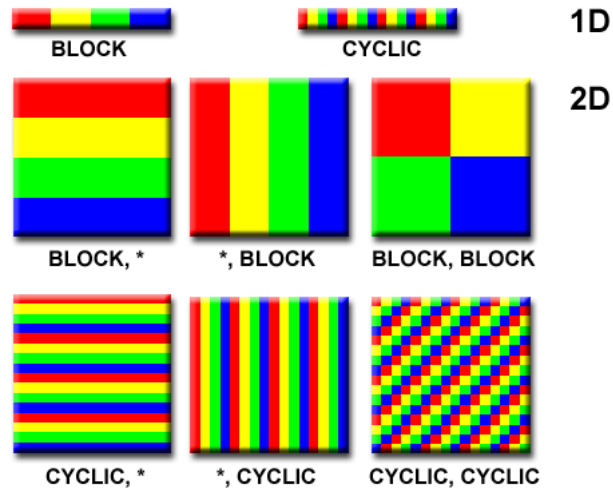
Remember **pthread_create()**?
Each thread executes a **copy**
of the parallel region

Synchronization

- High level:
 - critical
 - atomic
 - barrier
 - ordered
- Low level:
 - flush
 - locks

Synchronization: critical

```
float res;  
#pragma omp parallel  
{  
    id = omp_get_thread_num();  
    total = omp_get_num_threads();  
    for(i=id;i<n; i+total){  
        function-x(i);  
        #pragma omp critical  
        function-y(i);  
    }  
}
```



Hardcoded 1D-cyclic

Exclusive call to this function

Synchronization: atomic (advanced)

```
#pragma omp atomic  
x += tmp;
```

<https://stackoverflow.com/questions/7798010/what-is-the-difference-between-atomic-and-critical-in-openmp>

Worksharing

```
#pragma omp parallel
{
    id = omp_get_thread_num();
    total = omp_get_num_threads();
    for(i=id; i<n; i+total){
        function-x(i);}
}
```

Default fork-join is "Single Program Multiple Data" (SPMD), i.e., each thread runs the same copy of the parallel region and do different jobs based on your manually hardcoded branches using `omp_get_thread_num()`

But what you usually want is worksharing, i.e., you want openMP to help you share the work among all the threads for you

Manual 1D-cyclic



1D

Worksharing

```
#pragma omp parallel
{
    id = omp_get_thread_num();
    total = omp_get_num_threads();
    for(i=id; i<n; i+total){
        function-x(i);}
}
```

OpenMP constructs to help you do work sharing:

Loop
Section
Single
Task

Worksharing: parallel for

```
#pragma omp parallel
{
    int i, id, total;
    id = omp_get_thread_num();
    total = omp_get_num_threads();
    task-size = n / total;
    my-start = id * task-size;
    my-end = (id+1) * task-size;
    for(i=my-start;i<my-end; i++){
        function-x(i);
    }
}
```



Create a team of threads

```
#pragma omp parallel
{
    int i;
```

Use that team of threads

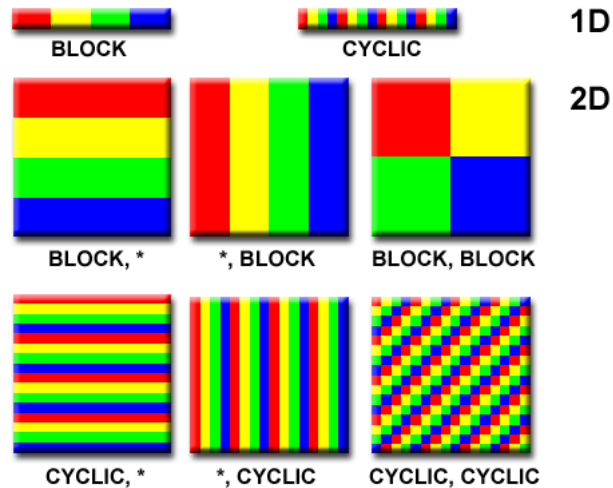
```
#pragma omp for
for(i=0;i<n; i++){
    function-x(i);
}
}
```

A normal forloop syntax now

- Variable i is private by default;
- explicitly we can write as
 - **#pragma omp for private (i)**
- Each thread has its local i value

Parallel for's schedule

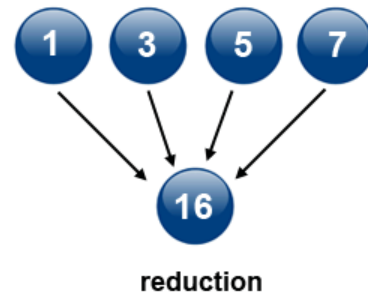
- Default is 'block'
- You can change it by writing:
 - `#pragma omp parallel for schedule(static, ...)`
 - i.e. cyclic (round-robin)
 - `#pragma omp parallel for schedule(dynamic, ...)`
 - = fixed-block, but a thread dynamically acquires the next block on free
 - ...



Reduction

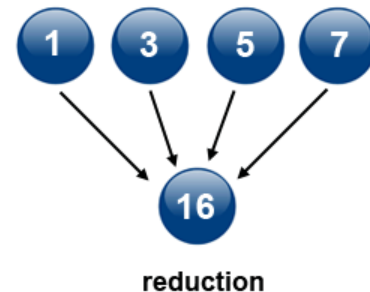
- Loop-carried dependence
 - E.g., New sum depends on previous sum
- For this example, you can change it be loop independent:
 - parallel for, local sum per thread → combining to a global sum
 - → The syntax is even easier if you know it is a **reduction** operation
 - Apply a **binary** operation (e.g., +) to a sequence of data to get a single result

```
int sum=0;  
for (i=0; i<n; i++)  
    sum=sum+A[i];
```



```
#pragma omp parallel for reduction(+: sum)  
for (i=0; i<n; i++)  
    sum=sum+A[i];
```

Reduction



- reduction (op : var)
 - A **local copy** of "var" is made for each thread
 - Local copies are reduced to a single value and "op" that with the original global value
- E.g.,
 - A local copy of "sum" is made for each thread
 - Local copies of "sum"s are reduced by "+" to the global "sum"

```
#pragma omp parallel for reduction(+:sum)
for (i=0; i<n; i++)
    sum=sum+A[i];
```

Task Pool

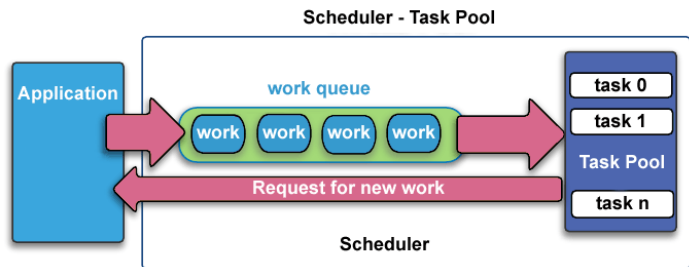
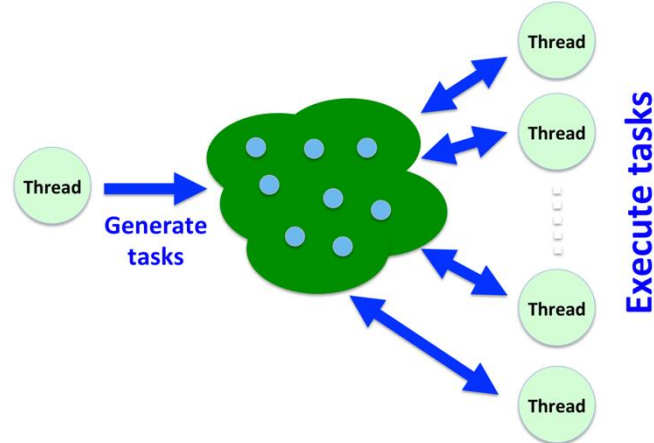
Developer specifies independent code block as task

```
#pragma omp task
```

```
{“this is my task”}
```

OpenMP runtime

- When a thread encounters a task construct, a new task is generated
- The runtime picks a task to execute



```
int main(int argc, char *argv[]) {
```

```
    #pragma omp parallel
```

```
{
```

```
    #pragma omp single    //this code block can only be run over once even omp  
    {                    parallel has spawned 400 threads
```

```
        printf("A ");
```

```
        #pragma omp task
```

```
            {printf("car ");}
```

```
        #pragma omp task
```

```
            {printf("race ");}
```

```
        #pragma omp taskwait
```

```
        printf("is fun to watch ");
```

```
    }
```

```
} // End of parallel region
```

```
printf("\n");return(0);
```

```
}
```

```
$ cc -xopenmp -fast hello.c  
$ export OMP_NUM_THREADS=2  
$ ./a.out  
$  
A car race is fun to watch  
$ ./a.out  
A car race is fun to watch  
$ ./a.out  
A race car is fun to watch  
$
```


References

- <https://computing.llnl.gov/tutorials/openMP/>
- <https://www.openmp.org/wp-content/uploads/omp-hands-on-SC08.pdf>
- <https://classroom.udacity.com/courses/ud281/>
- <http://parallelcomp.uw.hu>